

Task 2

Protocol Security

Reward: 20 Points

Mode: Group Work (max. 3 students)

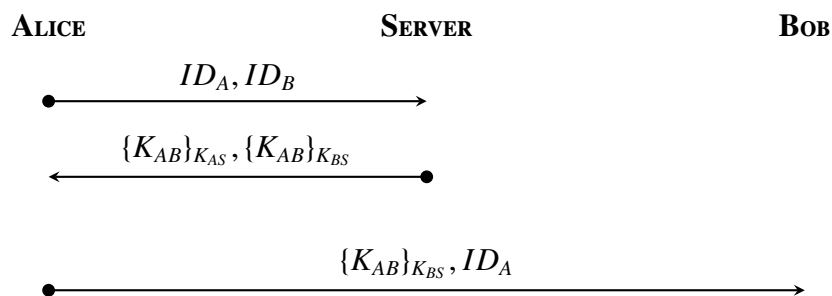
Due: 26.05.2026 -- 14:15

1 Pen & Paper

We start with a pen & paper task, before turning to a more practical task where you will work with a framework that implements a set of protocols in the second part of this assignment.

1.1 Authentication I

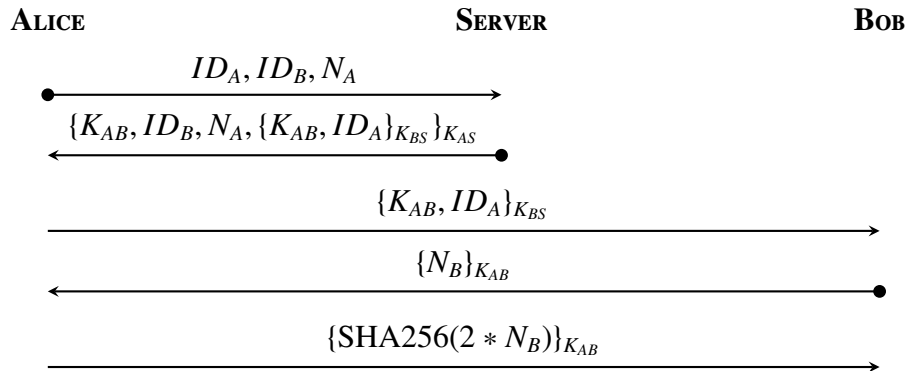
Given the protocol below, describe an attack that allows an adversary to manipulate the communication between Alice and Bob. The attack should enable the adversary to read and meaningfully replace all messages exchanged between Alice and Bob after successful authentication. Sketch the messages exchanged between the individual parties like in the given protocol.



1.2 Authentication II

The protocol below authenticates Alice towards Bob and vice versa through a mutually trusted server. The protocol leverages symmetric keys K_{PS} that are pre-shared between a party P and the server S .

Assume that the adversary observed a previous authentication session between Alice and Bob where they agreed on the session key K'_{AB} . Describe an attack on the protocol given below by which an adversary can convince Bob to use the old session key K'_{AB} , which is now known to the adversary. It is irrelevant here how exactly the adversary learned K'_{AB} . Outline the attack similar to the protocol given below.



2 Practical Tasks

In the next tasks, we will deal with the practical implementation of protocol attacks. We will provide you with a framework in which the protocols are implemented and in which you can intercept, modify, save and send messages. This section is divided into two parts.

2.1 Framework

The binaries provided in Moodle and the TCP proxy form the framework for this task. It is a TCP network that can be run entirely locally and without an internet connection. For this purpose, the parties in the network (Alice, Bob, and Co.) communicate with a server that routes the messages. You can think of the server as *the Internet*, while Alice and Co. are the clients, i.e., the users of the *Internet*.

A proxy (i.e., an intermediate station that forwards messages) is connected in front of the server. This is where you can connect to the network, so you have complete control over the entire network. A rough sketch of an example of the framework provided can be found in Figure 1. If Alice now wants to send the message *Hello, Bob* to Bob, she enters herself as the sender (*sender*), Bob as the recipient (*receiver*), and *Hello, Bob* as the content (*content*) in her message. Alice then sends this message to the proxy, about which Alice knows nothing; she only knows *the Internet*. You, the proxy, then drop the message or forward it (modified if necessary) to the server, whereupon the server forwards the message to the *receiver*, provided it is in the network. If the *receiver* is not in the network, an error message is displayed.

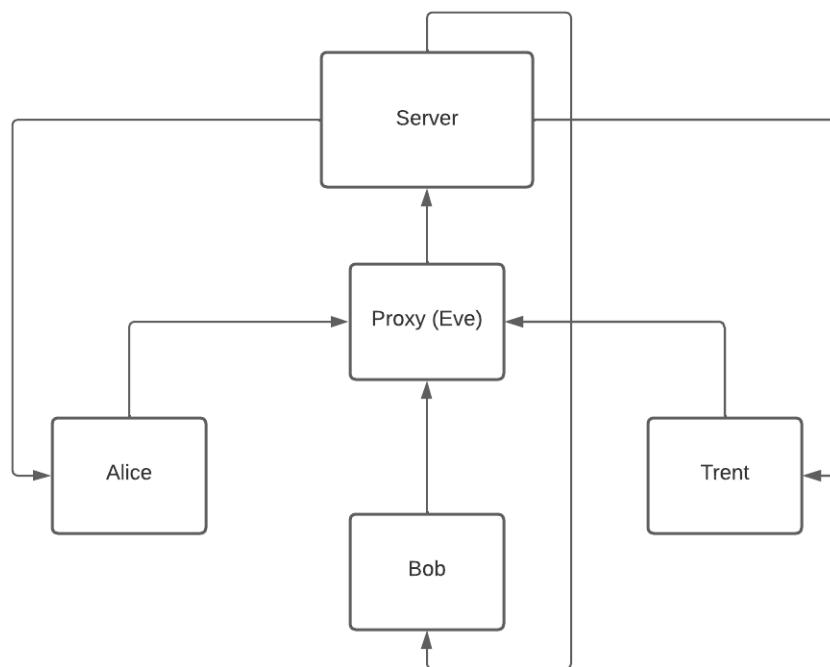


Figure 1: Network with exemplary parties Alice, Bob and Trent

2.1.1 Starting the Framework

The framework comprises:

- a) The network binary. This can be launched via the command line and accepts the flags listed below.

```
1 ./network --protocol=<PROTOCOL> (--server-port=<SERVER-PORT>) (--proxy-port=<PROXY-PORT>)
```

- `protocol` specifies the protocol to be executed. (see sub-tasks)
- (*Optional*, default is 8035) `server-port` specifies the port number on which the server should listen.
- (*Optional*, default is 8034) `proxy-port` specifies the port number on which the proxy should listen. An explicitly specified port number of 0 allows the protocol to be tested without a proxy.

- b) The TCP proxy (<https://github.com/ickerwx/tcpproxy>) to be used. In the folder `tcpproxy/proxymodules`, you'll find the file `syssec.py`. Here you can modify or drop packets. The proxy is started with the command:

```
1 python3 tcpproxy.py -ti 127.0.0.1 -tp <SERVER-PORT> -li 127.0.0.1 -lp <PROXY-PORT> -v -om  
   ↪ syssec
```

If the network binary was started without specifying server and proxy port flags, the proxy is thus started with:

```
1 python3 tcpproxy.py -ti 127.0.0.1 -tp 8035 -li 127.0.0.1 -lp 8034 -v -om syssec
```

Note that the proxy must be started before the network binary to ensure correct port interaction.

2.1.2 Proxy Modules

In the file `syssec.py`, you will find a sample implementation for dropping and modifying packets. In the function `execute`, you receive the data (`data`), which you can then process and return.

Thus, to drop packets with an `id` of 1, use:

```
1 # drop packet #1  
2 if data_json["id"] == 1:  
3     return bytes()
```

To reflect a packet with `id = 2`, use:

```
1 # reflect packet #2 back to the sender  
2 if data_json["id"] == 2:  
3     original_sender, original_receiver = data_json["sender"], data_json["receiver"]  
4     data_json["sender"], data_json["receiver"] = original_receiver, original_sender  
5     data = json.dumps(data_json)  
6     return data + "\n"
```

Note that the delimiter `\n` must be appended. If omitted, your message won't be recognized by the server.

2.2 Notes

- A flag does not guarantee the correct solution to the task. Think about whether the chosen attack on the protocol makes sense in theory. If it does **and** a flag is shown to you, the correct solution was likely found. The flag is therefore only an indicator. Extracting the flag from the binary will **not** help you solve the task.
- It is **not** required to bring your own client into the network. All tasks can be solved solely by modifying or dropping packets, as described above.
- A DoS (Denial of Service) attack is **not** necessary.
- Modifying handshakes is **not** required. Handshakes are used to register a party in the network and must not be altered.

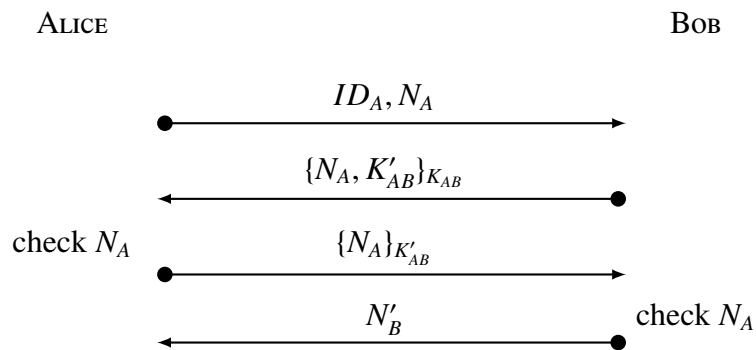
2.3 Tasks

In the following tasks, you will identify and implement attacks on various protocols using the framework. The framework provides implementations for all protocols.

Note: The attacker EVE is a legitimate part of the network. If there is a trusted third-party TRENT, Eve shares a symmetric long-term key K_{ET} with it. Note, however, that you do not need to perform any encryption or decryption yourself in these tasks.

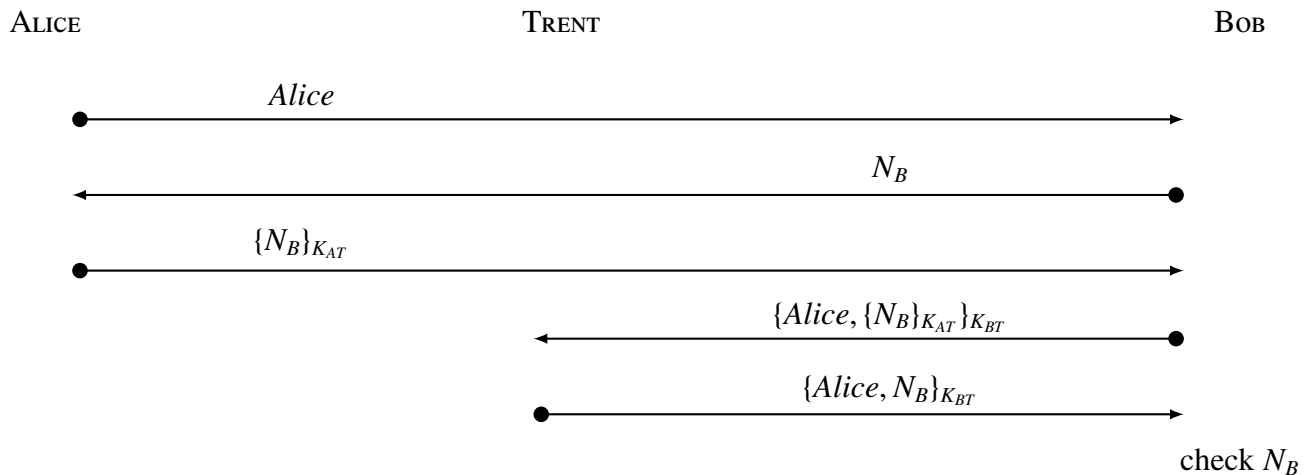
2.3.1 Key Establishment

This protocol is used to establish a session key K'_{AB} between two parties without interacting with a third party. Here, the key K_{AB} is a long-term key shared between ALICE and BOB. Your task is to develop an attack that allows the attacker to impersonate BOB to ALICE. The protocol can be executed with the command `./network --protocol=slke`.



2.3.2 Authentication with Trusted Party

In this scenario, a client ALICE uses their provider TRENT to access the network of a service provider BOB. With the help of the protocol, ALICE authenticates to BOB via TRENT as a trusted third party. ALICE and BOB trust each other. Both parties share a symmetric key with TRENT (K_{AT} for ALICE and K_{BT} for BOB). The goal of the attacker is to impersonate ALICE to BOB. The attacker should do more than merely act as a relay. The protocol can be executed with the command `./network --protocol=authentication`.



Deliverables

Please note that **you must have joined a group (1-3 students) via the “group selection” in Moodle** in order to submit the assignment.

- The submission takes place within a **single** PDF file, preferably in $\text{\LaTeX}$ with the available solution template. However, it is also possible to create a PDF with Word (or similar) and enter the protocol sequences there.
- **Handwritten submissions will not be accepted.**

For full points, the following aspects are required for a solution:

- Schematic protocol flows with the adversary and marked changes made by the adversary. Note that a description of why an attack is possible, how it is carried out, and what the result is for the adversary may help us award you partial points if your solution is incorrect.
- For each protocol in Task 2.3, a Python file with your sufficiently and appropriately commented code must be submitted---the code snippets in 2.1.2 are a good example for sufficient and meaningfully commented code. The code must run directly with the provided version of `tcp-proxy` under Python 3.10.

In total, three files must be submitted ($2 \times$ code and $1 \times$ PDF). To make drawing the protocol flows in $\text{\LaTeX}$ easier for you, we have provided the corresponding code **in Moodle**.

Plagiarism & Use of Generative AI

In this course, we respect other people's work and do not tolerate plagiarism. That means you should always give credit to the original authors if you:

- use someone else's work (whether you copy it directly, almost directly, or rewrite it in your own words),
- reuse parts of someone else's work, such as equations, figures, or tables.

We strongly encourage you to work on this assignment without using generative AI tools (like large language models), so you can build your own understanding of the material. Please, **in all cases**, state clearly how you used such tools:

- If you only used them for small edits (e.g., grammar or wording), add a note like: “This assignment was edited for grammar using [Tool Name].”
- If you did not use any AI tools, add: “This assignment was completed without the use of AI tools.”
- If you use AI for more than small edits, please specify in detail which parts of the solutions are generated by the AI tool and which steps you took to ensure the correctness of the output.
- Work that relies on AI to generate the major part of the submission, e.g., full scripts or key parts of the solution, will not be accepted.

If you use Stack Overflow or similar resources, make sure to clearly cite any code you did not write yourself.